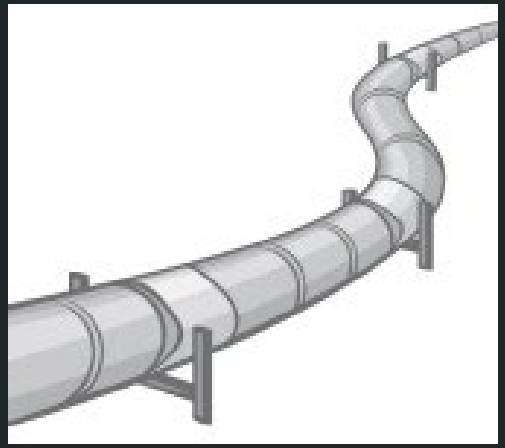
Processor Pipelining

CS 2410

Wonsun Ahn

University of Pittsburgh

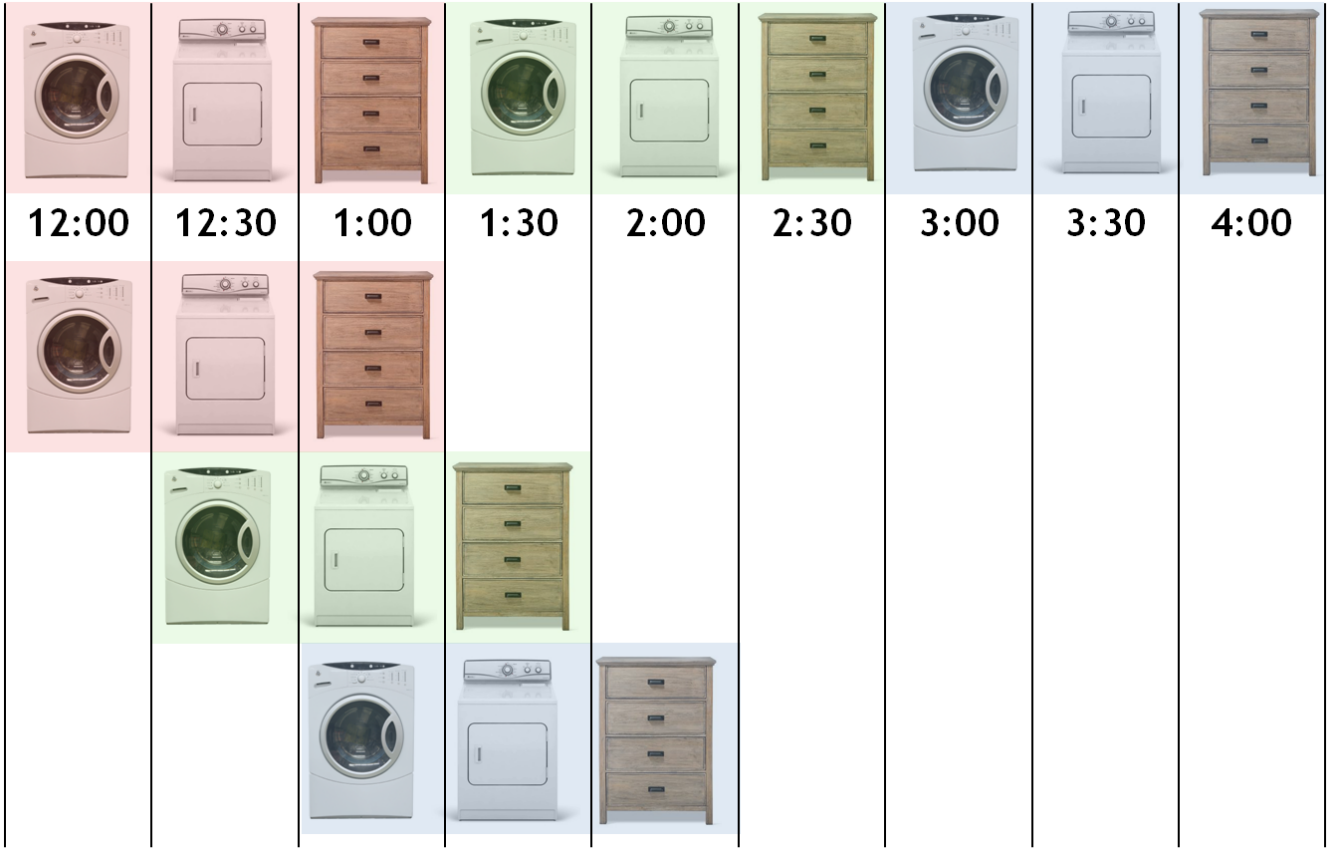
Pipelining Basics



University of Pittsburgh

Improving Washer / Dryer / Closet Utilization

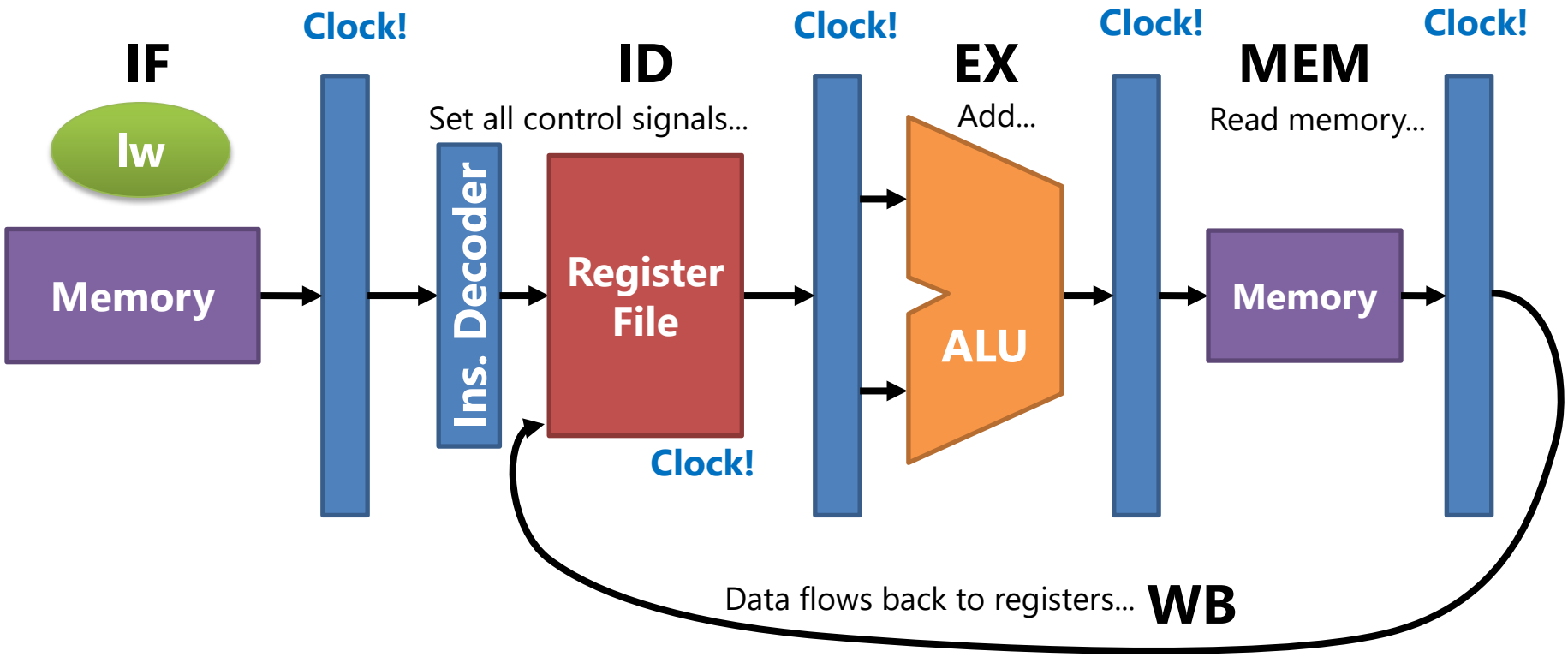
- If you work on loads of laundry one by one, you only get ~33% utilization
- If you form an “assembly line”, you achieve ~100% utilization!



University of Pittsburgh

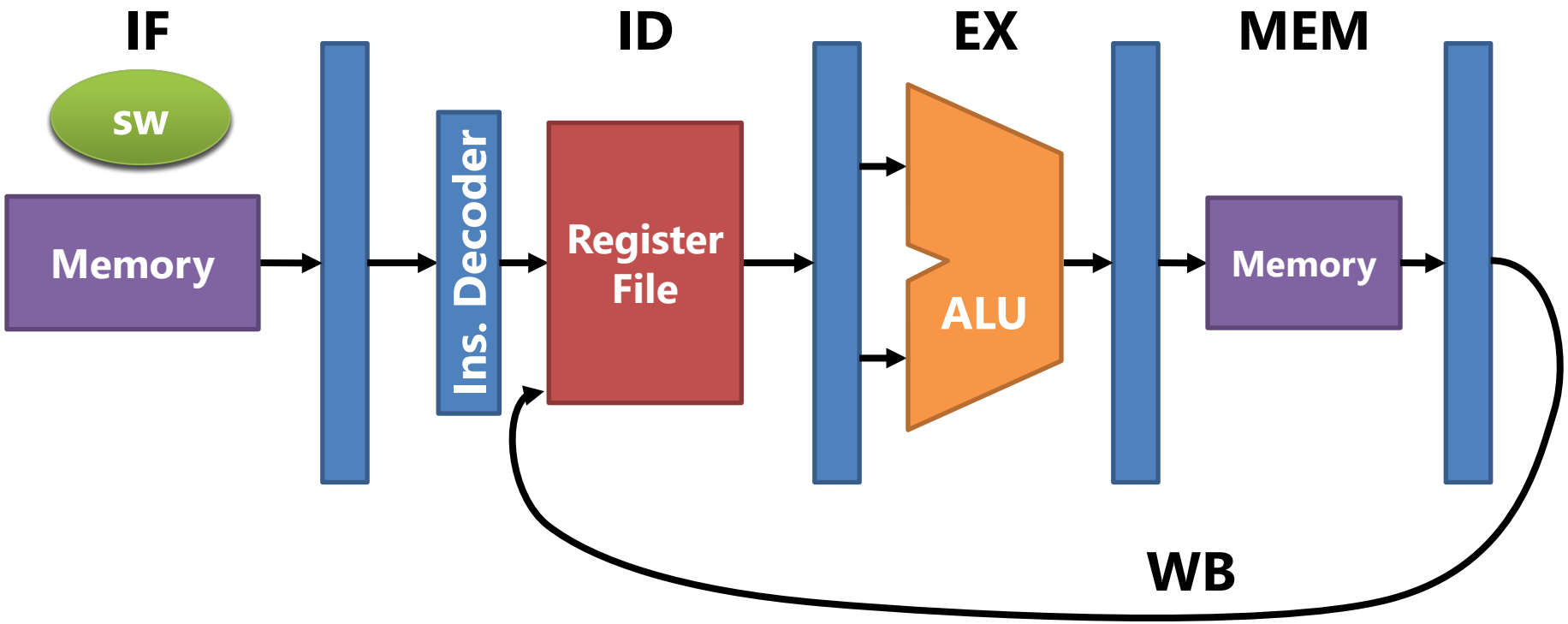
Multi-cycle instruction execution

- Let's watch how an instruction flows through the datapath.



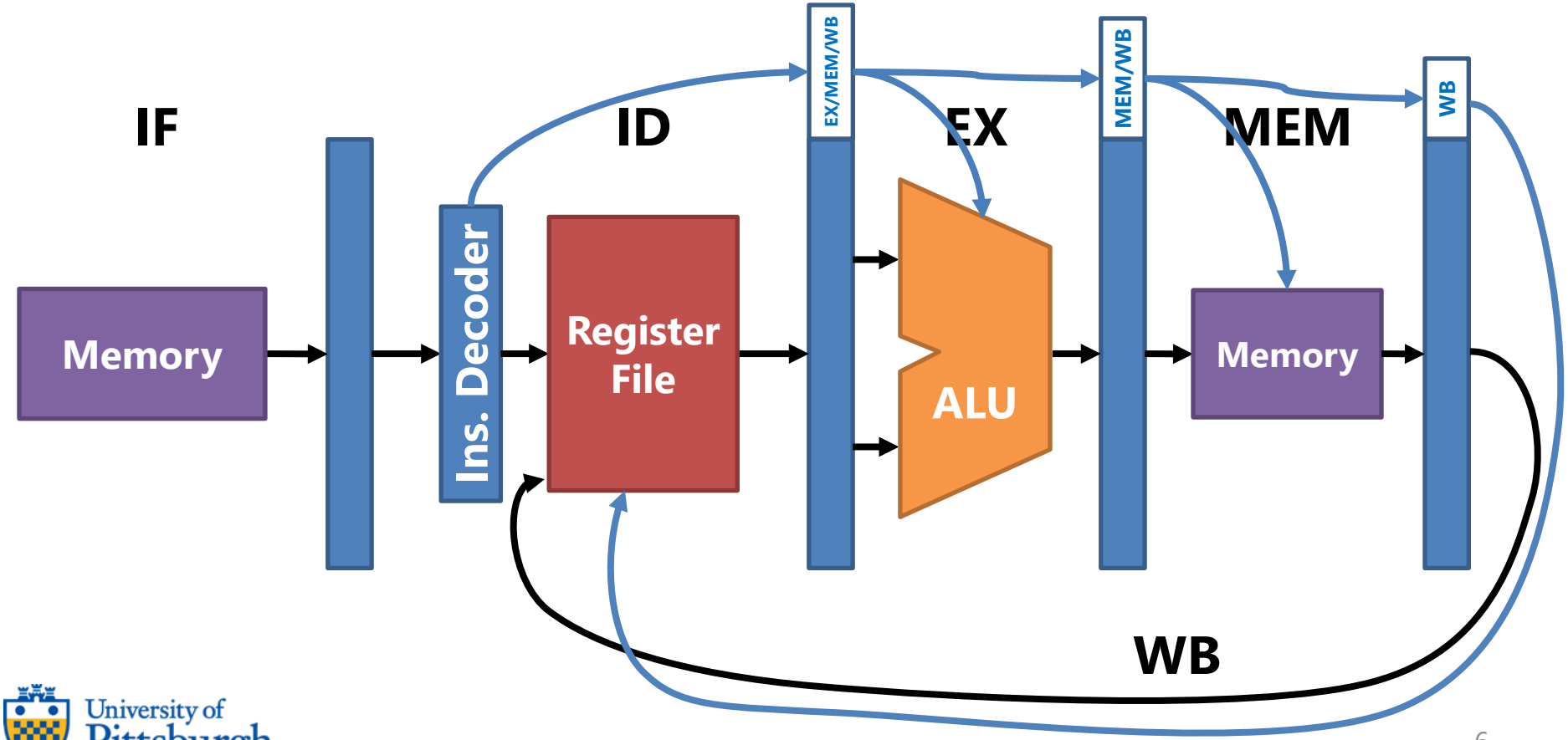
Pipelined instruction execution

- Pipelining allows one instruction to be fetched each cycle!



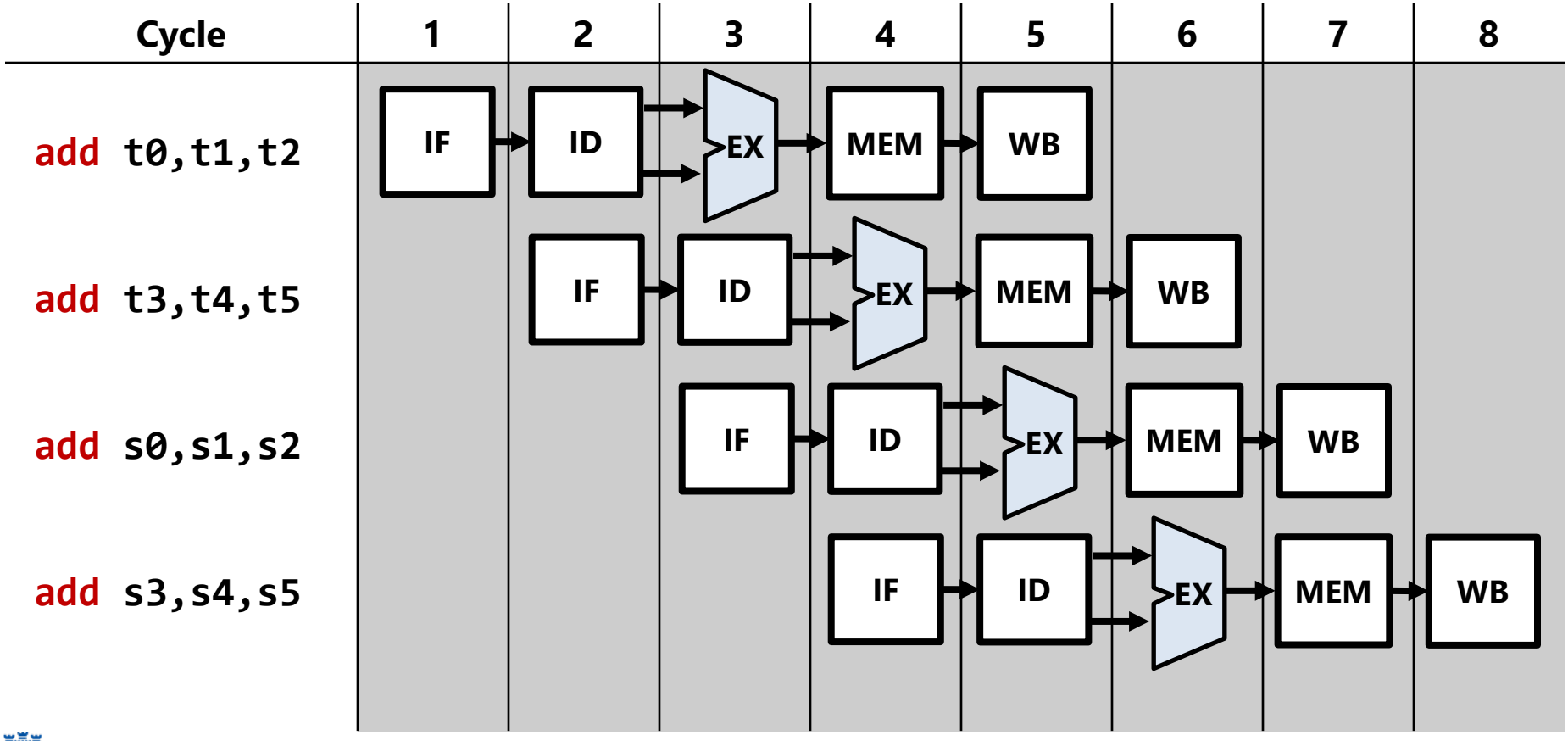
How about the control signals?

- A new instruction is decoded at every cycle!
- Control signals must be passed along with the data at each stage



Pipelining Timeline

- This type of parallelism is called *pipelined parallelism*.



A Pipelined Implementation is even Faster!

- Now every instruction takes the same number of cycles
  - **lw** takes 5 cycles: IF/ID/EX/MEM/WB
  - **add** takes 5 cycles: IF/ID/EX/---/WB
  - **sw** takes 5 cycles: IF/ID/EX/MEM/---

- If each stage takes *1 ns* each:

Q) Given 100 instructions, the average instruction execution time is?

A) (*5 ns* + *99 ns*) / 100 = *1.04 ns*
  - A ~**5X** speed up from single cycle!

Pipelined vs. Multi-cycle vs. Single-cycle

- What happened to the three components of performance?

$$\frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

Architecture	Instructions	CPI	Cycle Time (1/F)
Single-cycle	Same	1	5 ns
Multi-cycle	Same	4~5	1 ns
Pipelined	Same	1	1 ns

- Compared to single-cycle, pipelining improves clock cycle time
  - Or in other words CPU ***clock frequency***
  - The deeper the pipeline, the higher the frequency will be

** Caveat: latch delay and unbalanced stages can increase cycle time*

Pipeline Hazards

University of Pittsburgh

Pipeline Hazards

- For pipelined CPUs, we said CPI is practically 1
  - But that depends entirely on having the pipeline filled
  - In real life, there are ***hazards*** that prevent 100% utilization

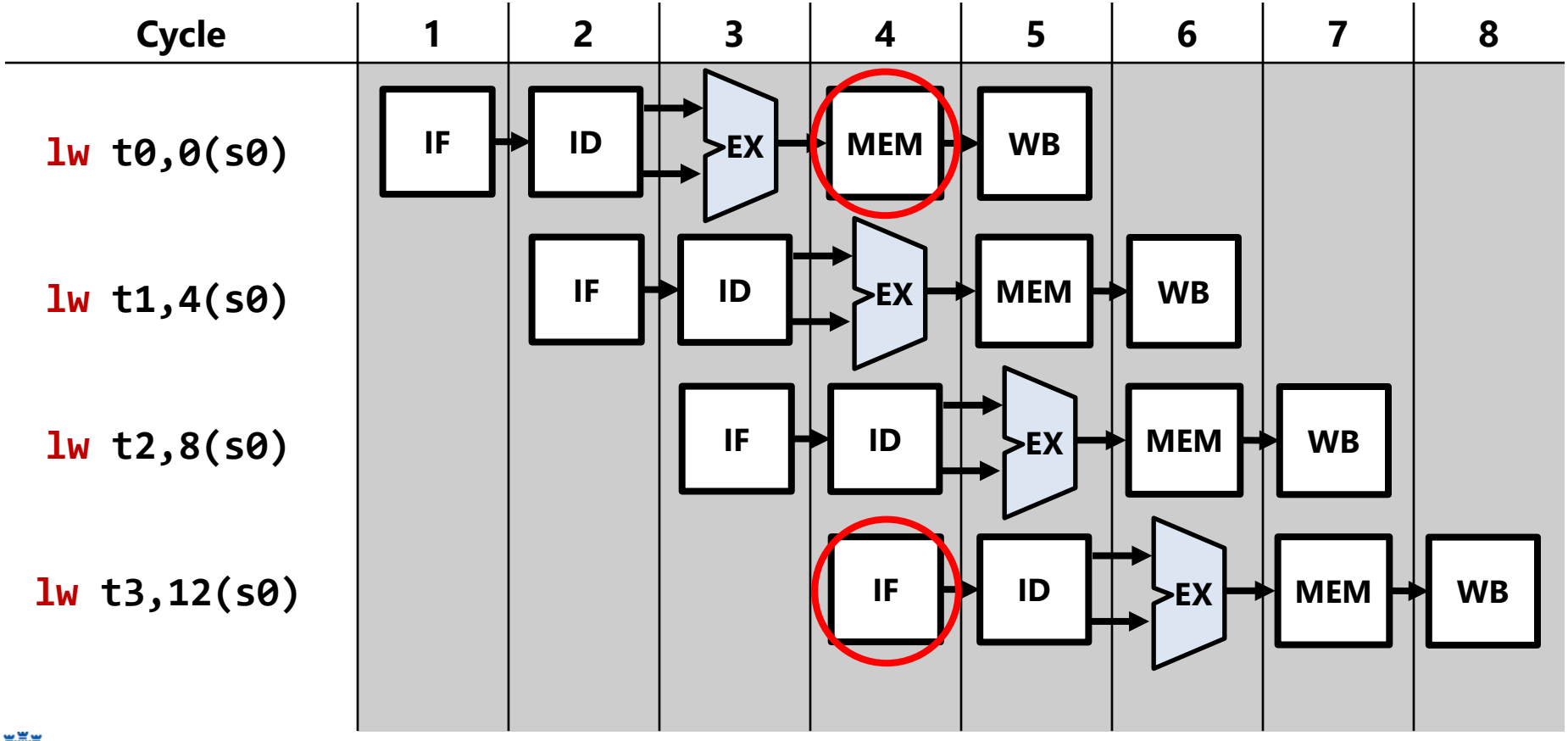
- ***Pipeline Hazard***
  - When the next instruction cannot execute in the following cycle
  - Hazards introduce ***bubbles*** (delays) into the pipeline timeline

- Architects have some tricks up their sleeves to avoid hazards

- But first let's briefly talk about the three types of hazards: *Structural hazard*, *Data hazard*, *Control Hazard*

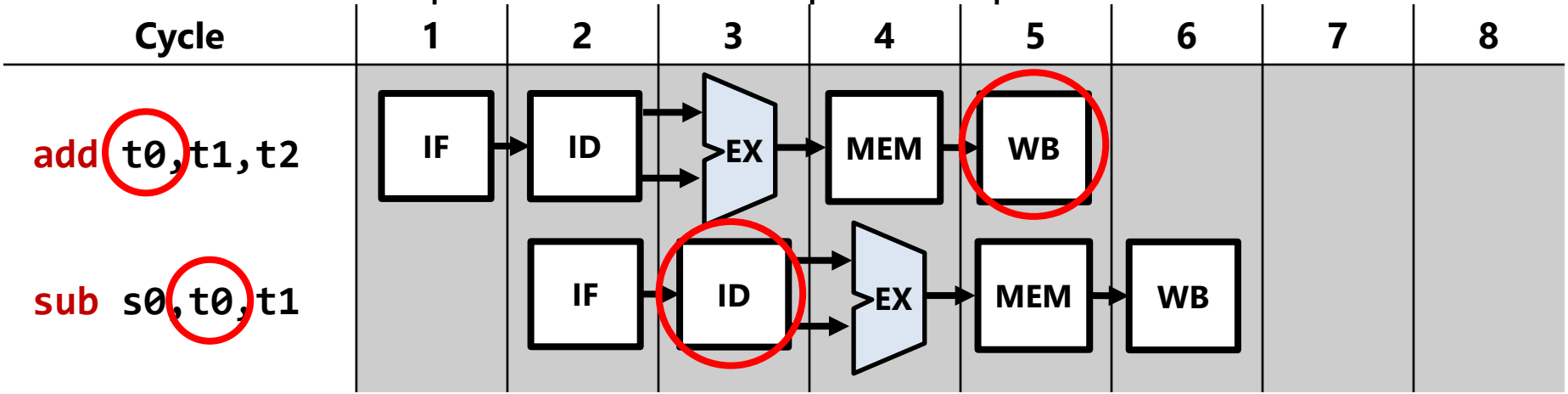
Structural Hazards

- Two instructions need to use the same hardware at the same time.



Data Hazards

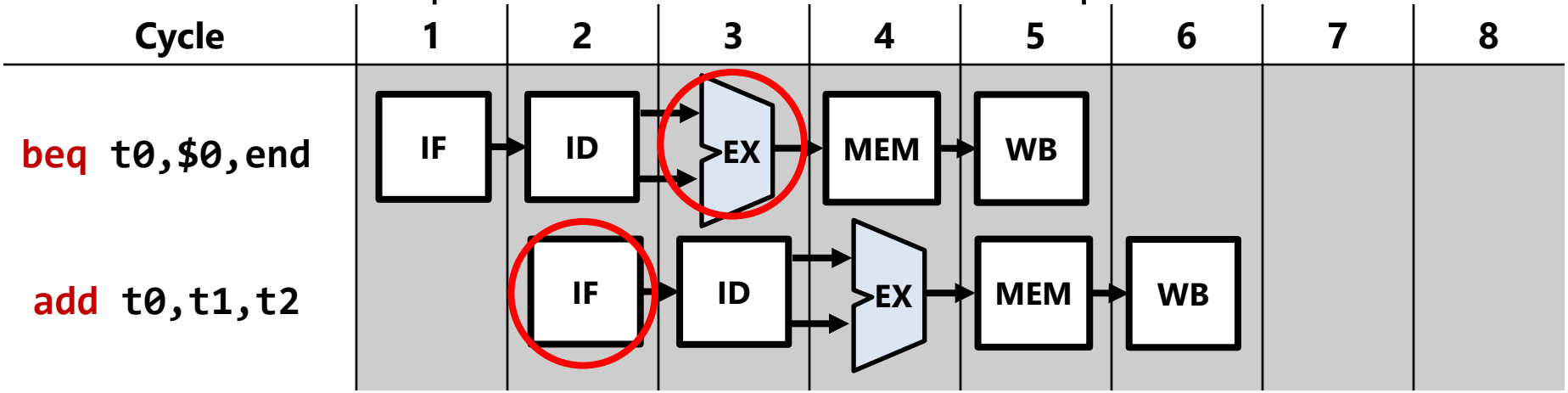
- An instruction depends on the output of a previous one.



- **sub** reads in **`t0`** before **add** has had a chance to write it back!

Control Hazards

- An instruction depends on branch outcome of previous instruction.



- **add** (PC+4) is fetched before **beq** branch outcome is known!

Dealing with Hazards

- Pipeline must be controlled so that hazards don't cause malfunction

- Who is in charge of that? You have a choice.

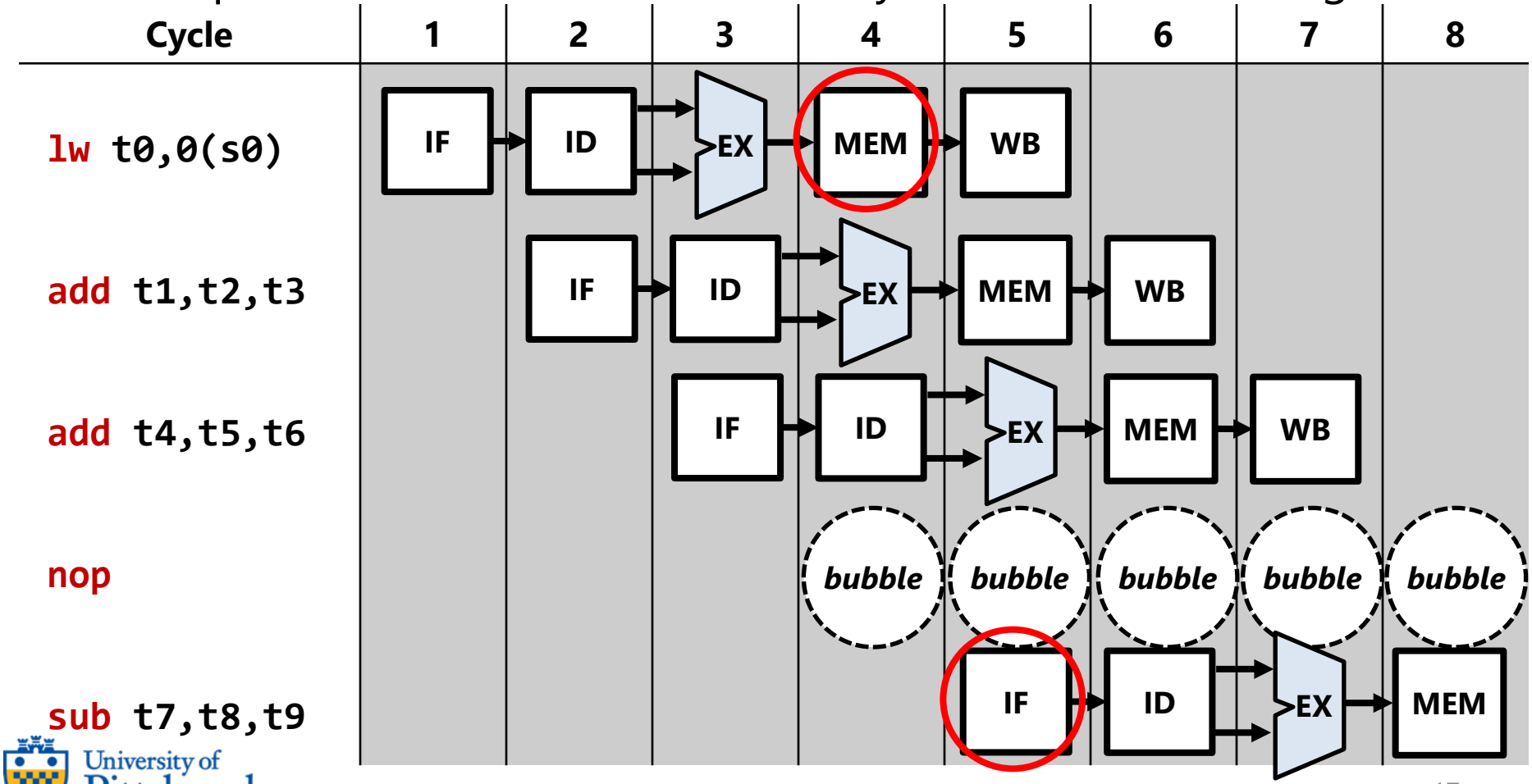
1. Compiler can avoid hazards by inserting nops
   - Insert nops where compiler thinks a hazard would happen
   - Nops flow through the pipeline not doing any work

2. CPU can internally avoid hazards using a ***hazard detection unit***
   - If structural/data hazard, pipeline ***stalled*** until resolved
   - If control hazard, pipeline ***flushed*** of wrong path instructions

Dealing with Hazards Using Compiler Scheduling

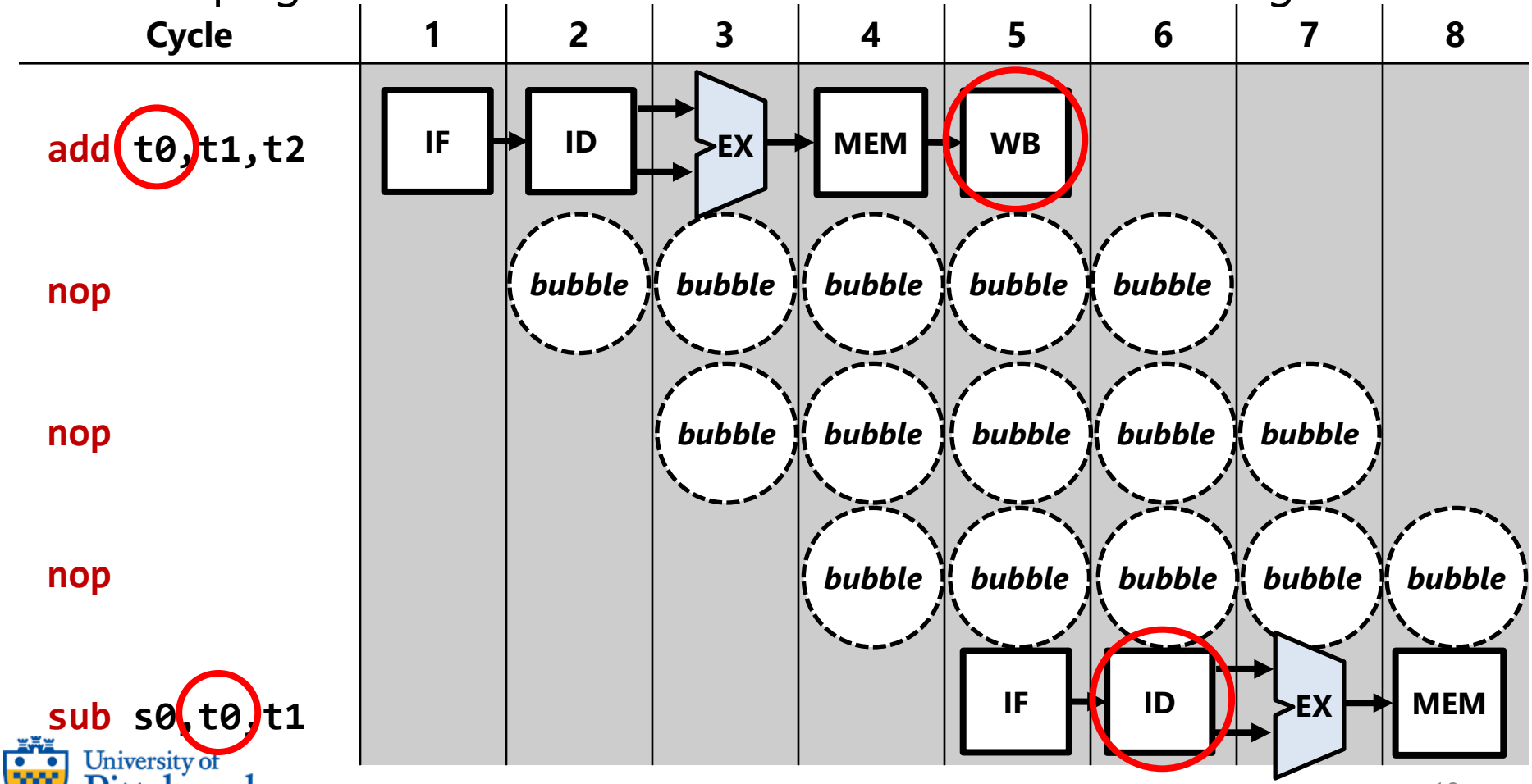
Compiler avoiding a structural hazard

- The nop avoids simultaneous access by the MEM and IF stages



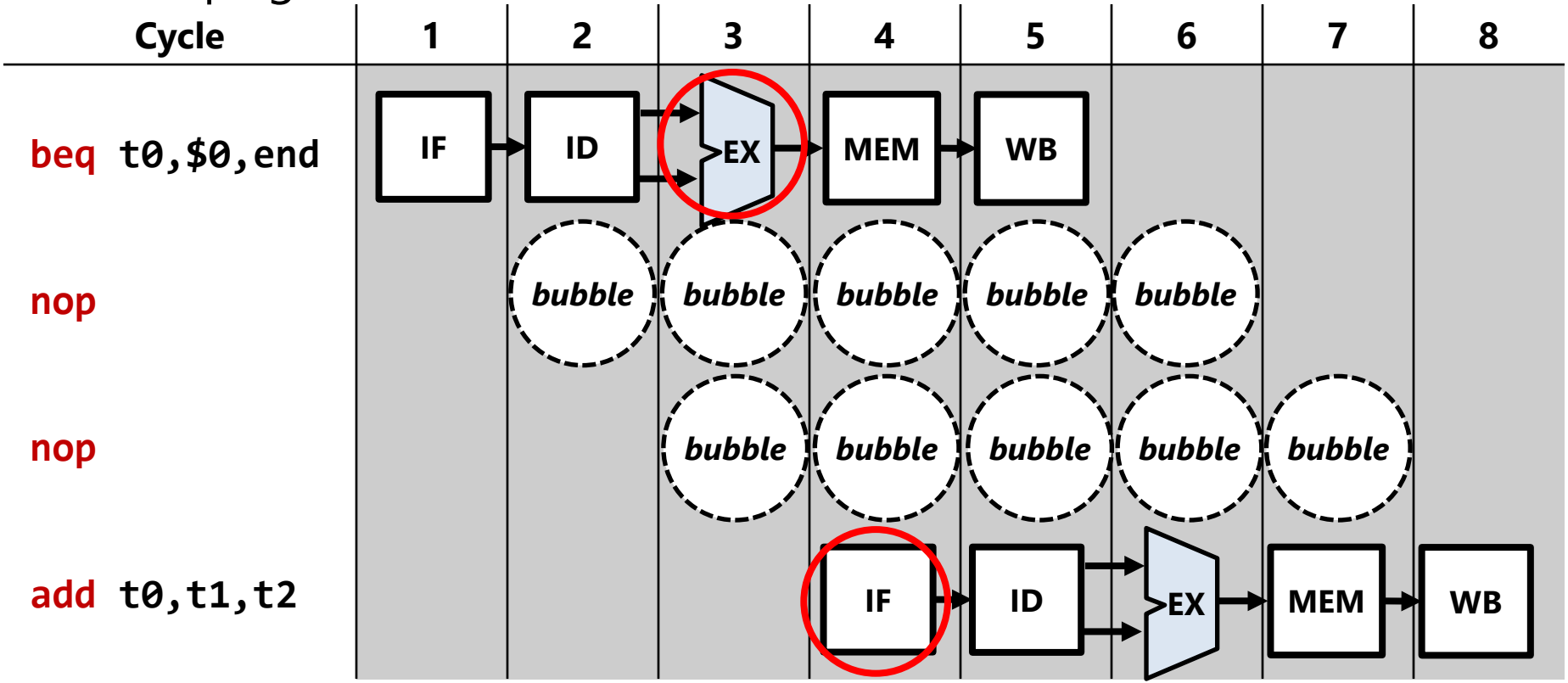
Compiler avoiding a data hazard

- The nops give time for **`t0`** to be written back before being read



Compiler avoiding a control hazard

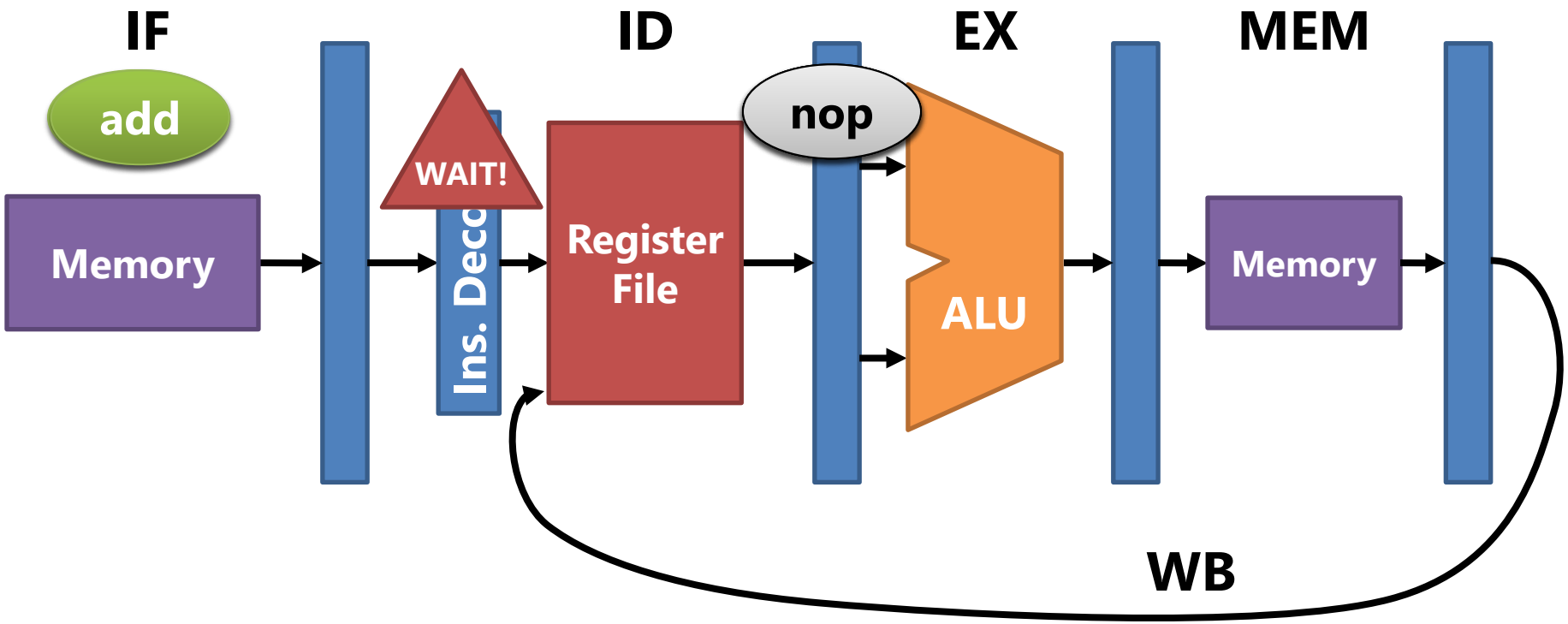
- The nops give time for condition to resolve before instruction fetch



Dealing with Hazards Using Hardware Scheduling

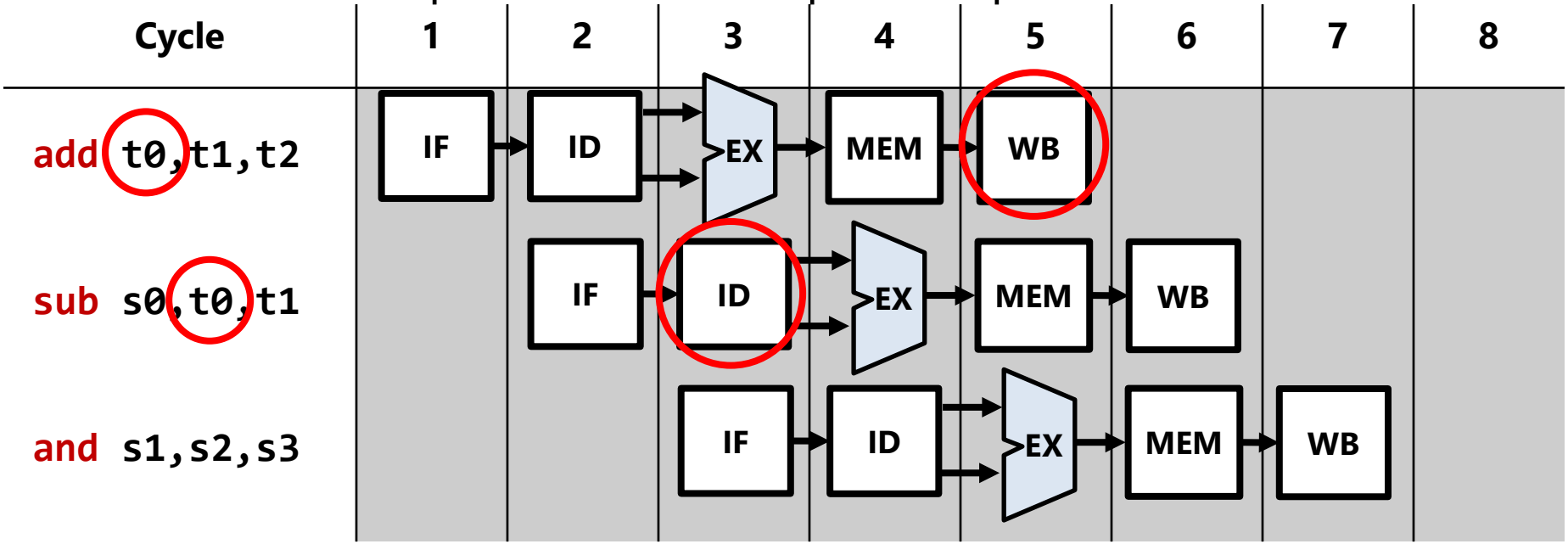
Hardware avoiding a data hazard

- Suppose we have an **add** that depends on an **lw**.



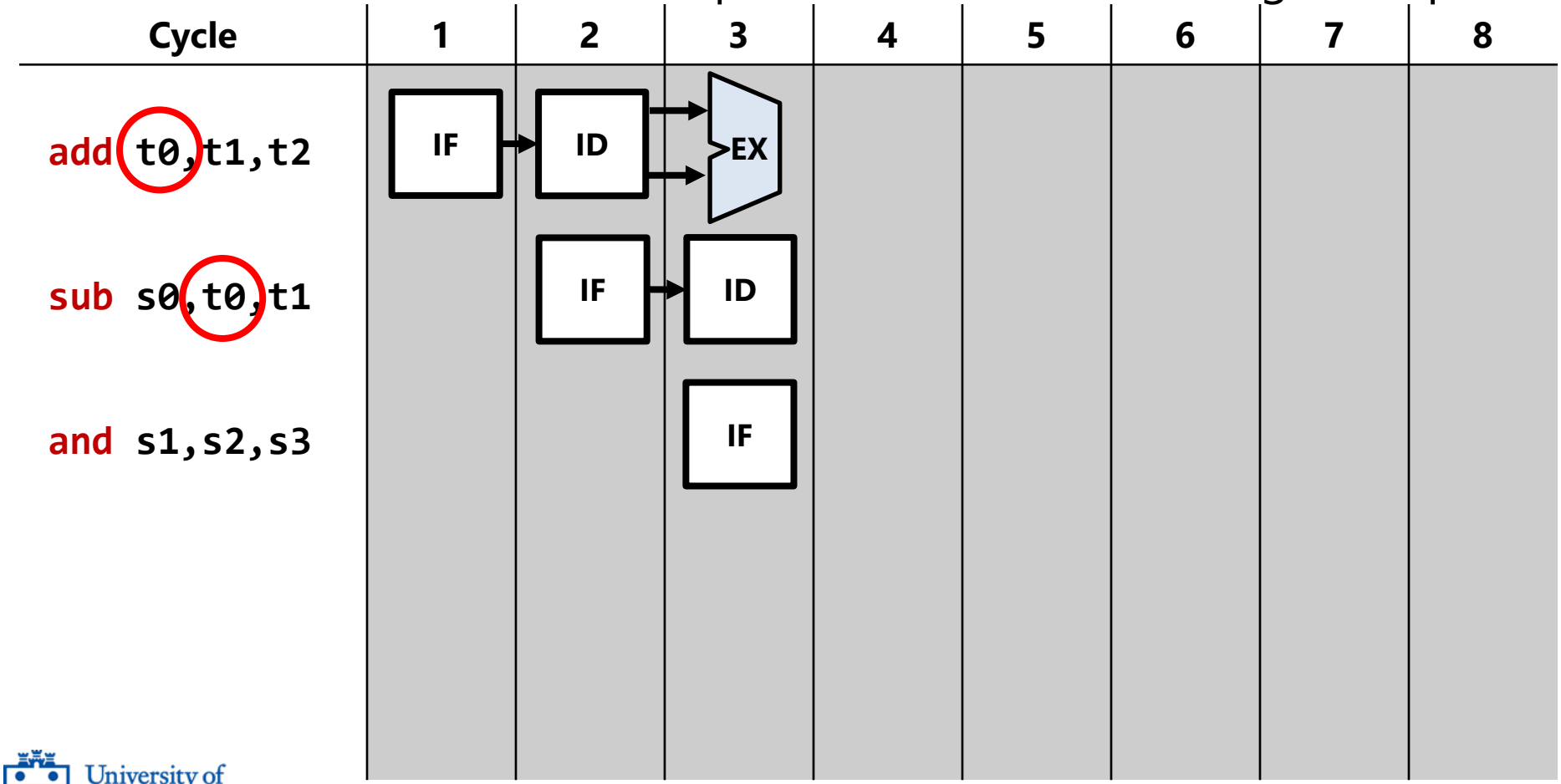
Hardware avoiding a data hazard

- An instruction depends on the output of a previous one.



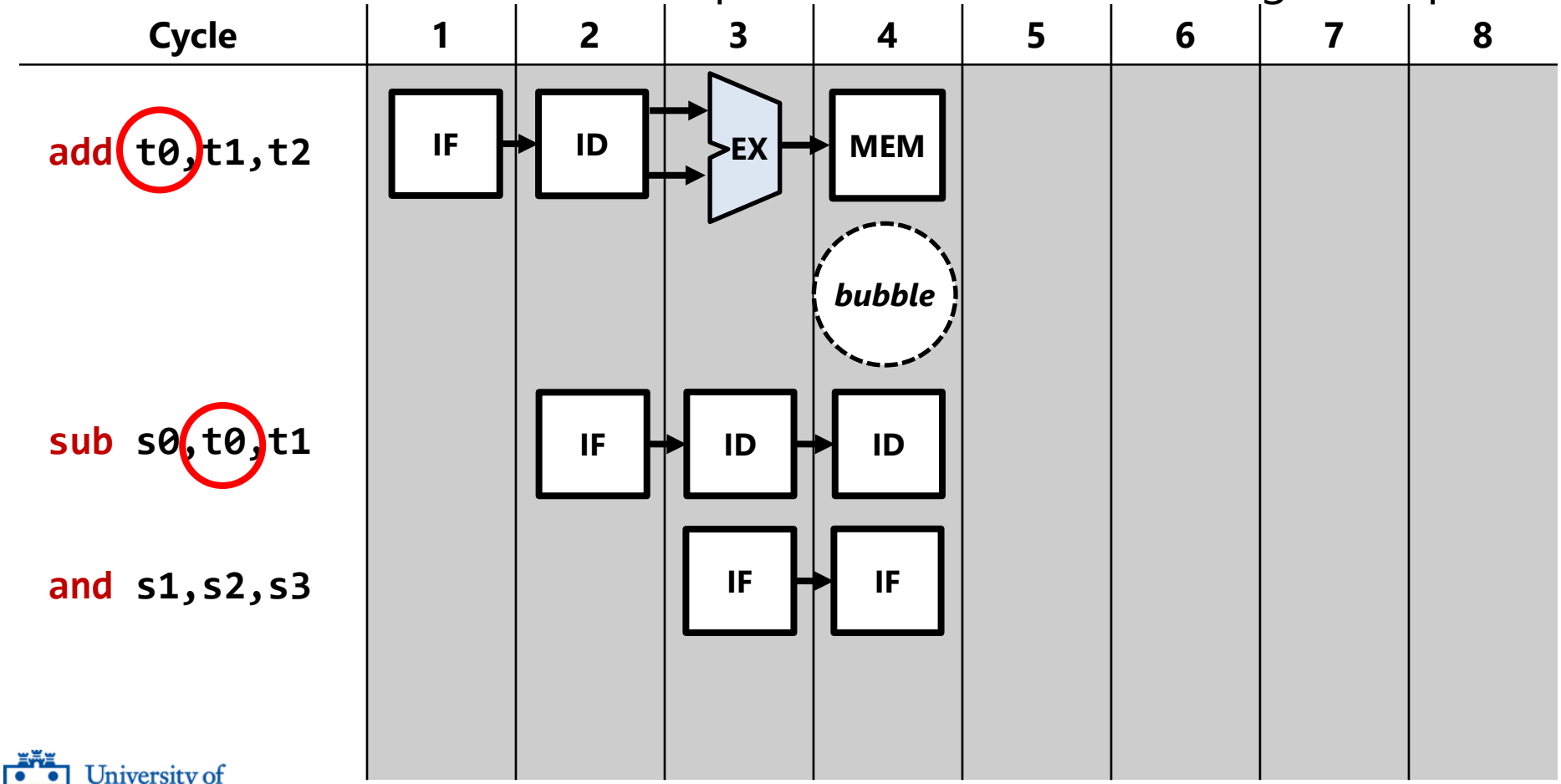
Hardware avoiding a data hazard

- **sub** must wait until **add**'s WB phase is over before doing its ID phase



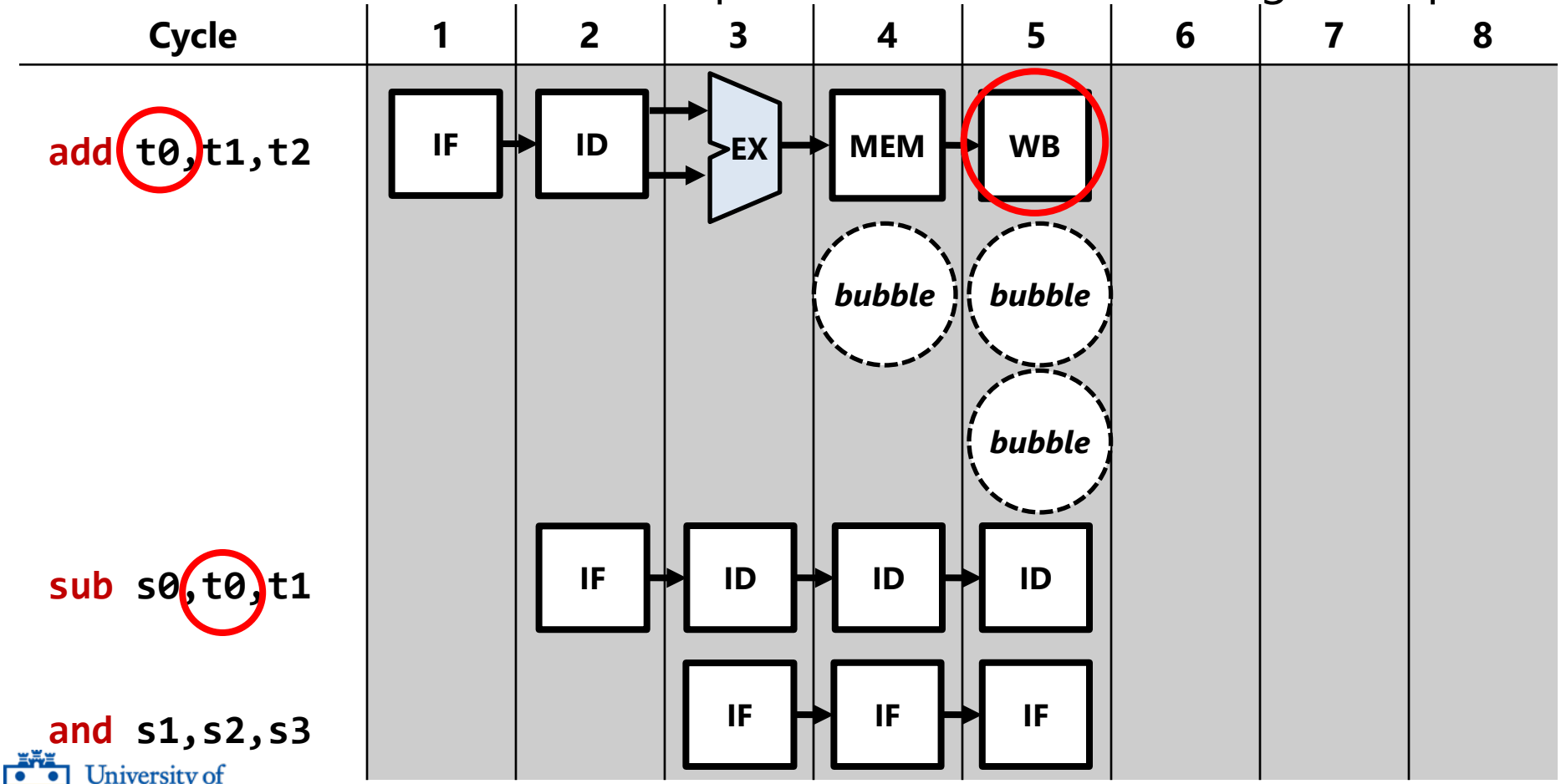
Hardware avoiding a data hazard

- **sub** must wait until **add**'s WB phase is over before doing its ID phase



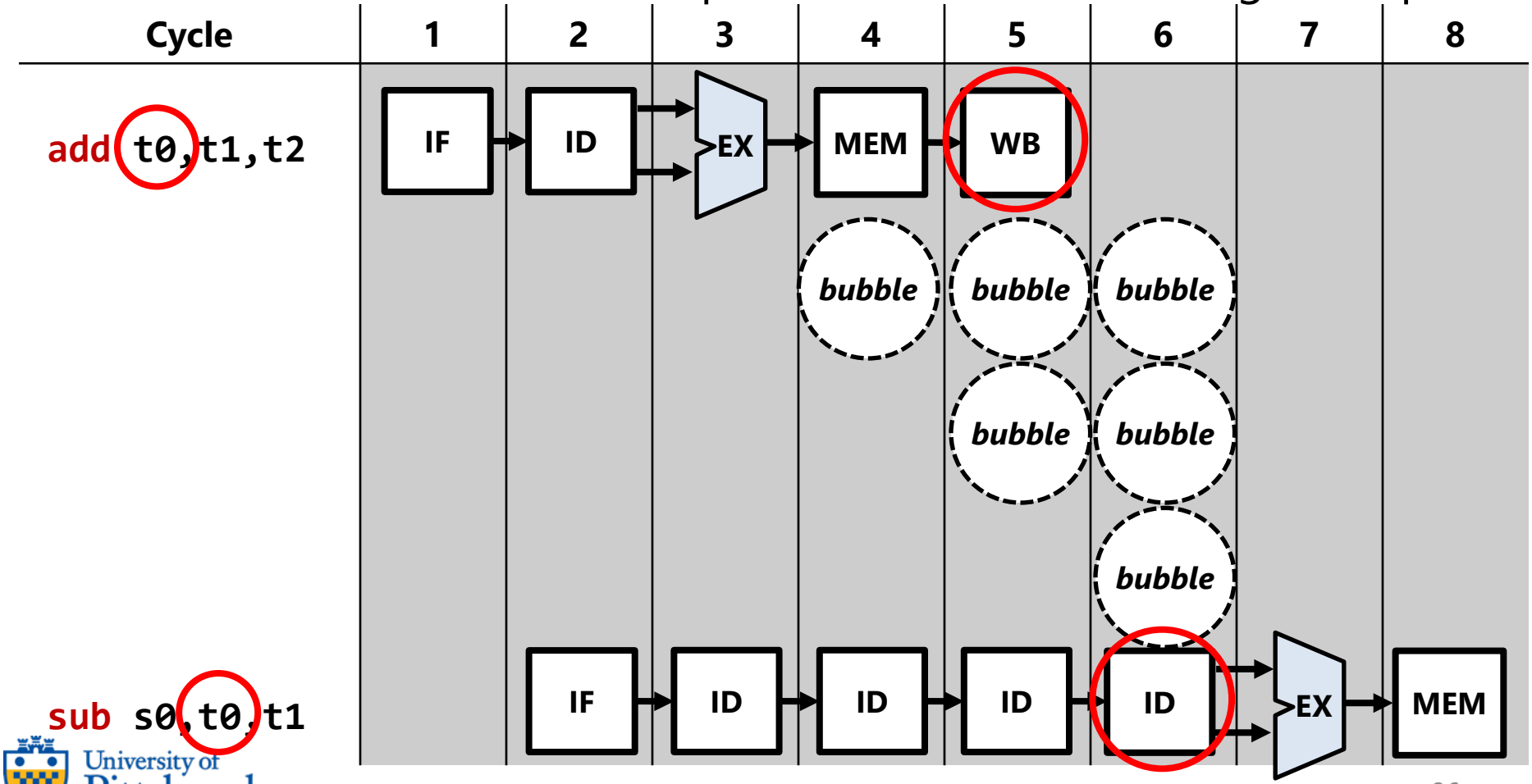
Hardware avoiding a data hazard

- **sub** must wait until **add**'s WB phase is over before doing its ID phase

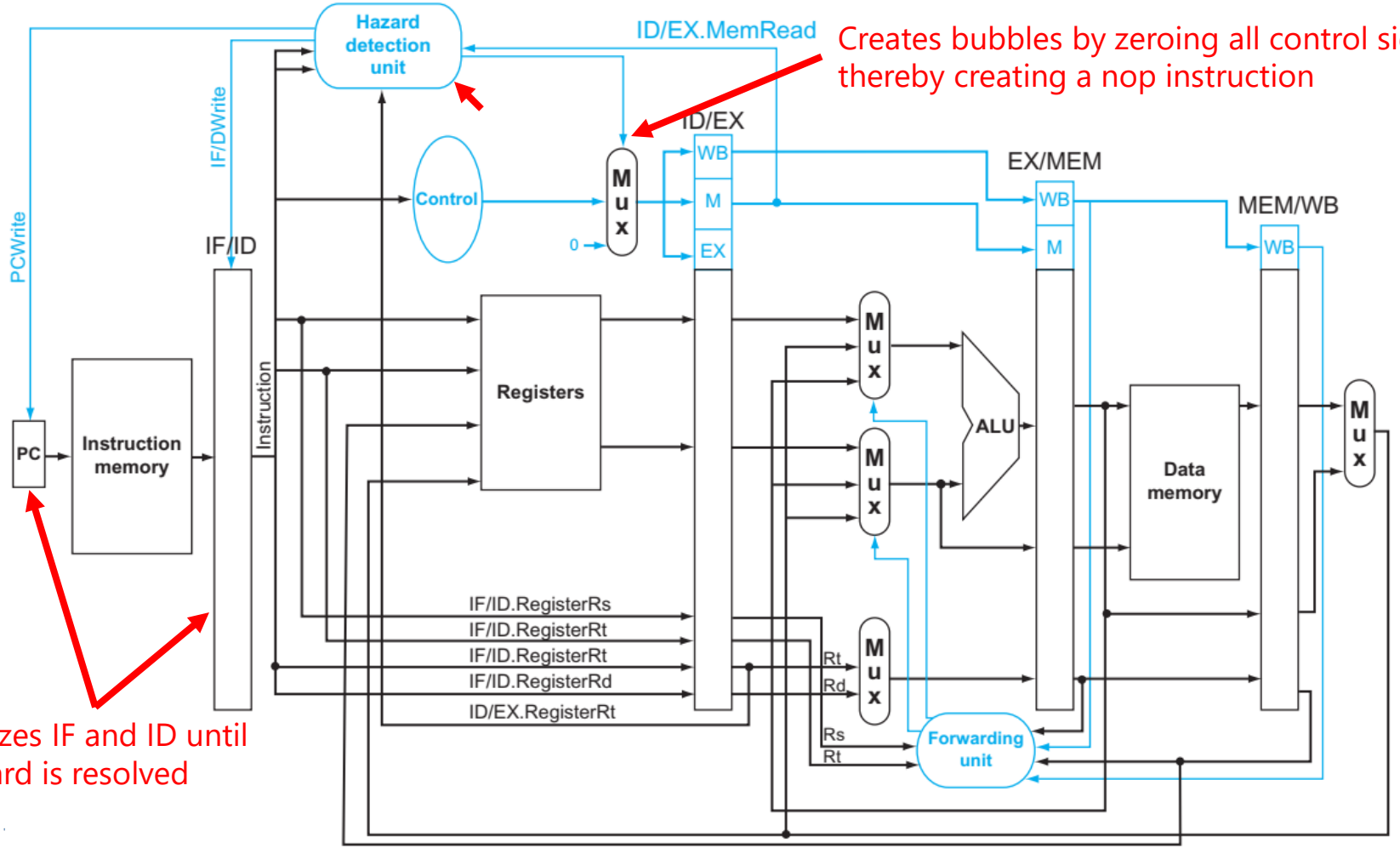


Hardware avoiding a data hazard

- **sub** must wait until **add**'s WB phase is over before doing its ID phase



Hazard Detection Unit (HDU) on data hazard



Structural / Data Hazards cause stalls

- If HDU detects a structural/data hazard during decoding, it:
  - **Freezes IF stage** (by stopping clock signal into PC)
  - **Freezes ID stage** (by stopping clock signal into IF/ID latch)
  - **Starts inserting bubbles into EX stage**
    - By passing nop to ID/EX latch (select 0 signal using mux)

- When structural or data hazard is resolved
  - HDU unfreezes IF and ID stages and stops inserting bubbles

- But what about control hazards?
  - Instructions in wrong path are already in pipeline!
  - Need to ***flush*** these instructions

Control Hazard Example

- Supposed we had this for loop followed by printf("done"):

```
for(s0 = 0 .. 10)
    print(s0);
```

```
printf("done");
```

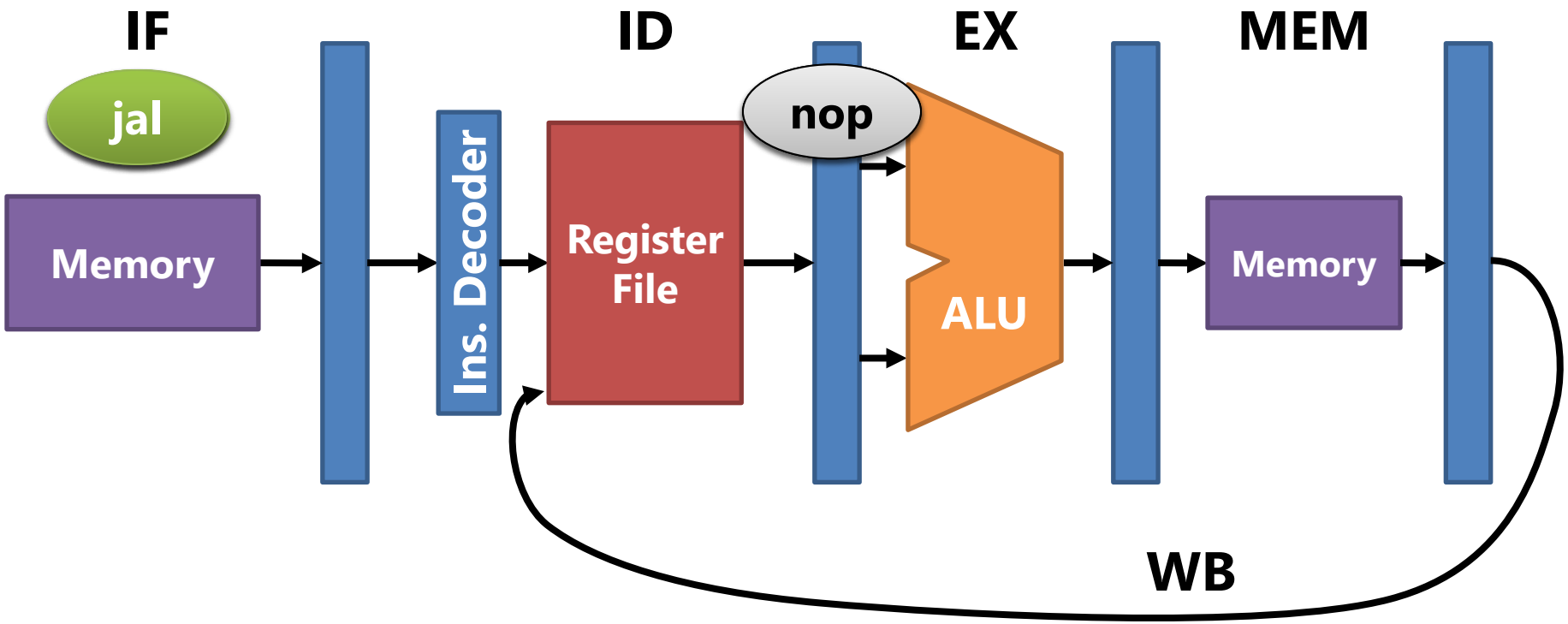
```
    li   s0, 0
top:
    move a0, s0
    jal  print
    addi s0, s0, 1
    blt  s0, 10, top
```

```
    la   a0, done_msg
    jal  printf
```

By the time **s0, 10** are compared at **blt** EX stage, the CPU would have already fetched **la** and **jal**!

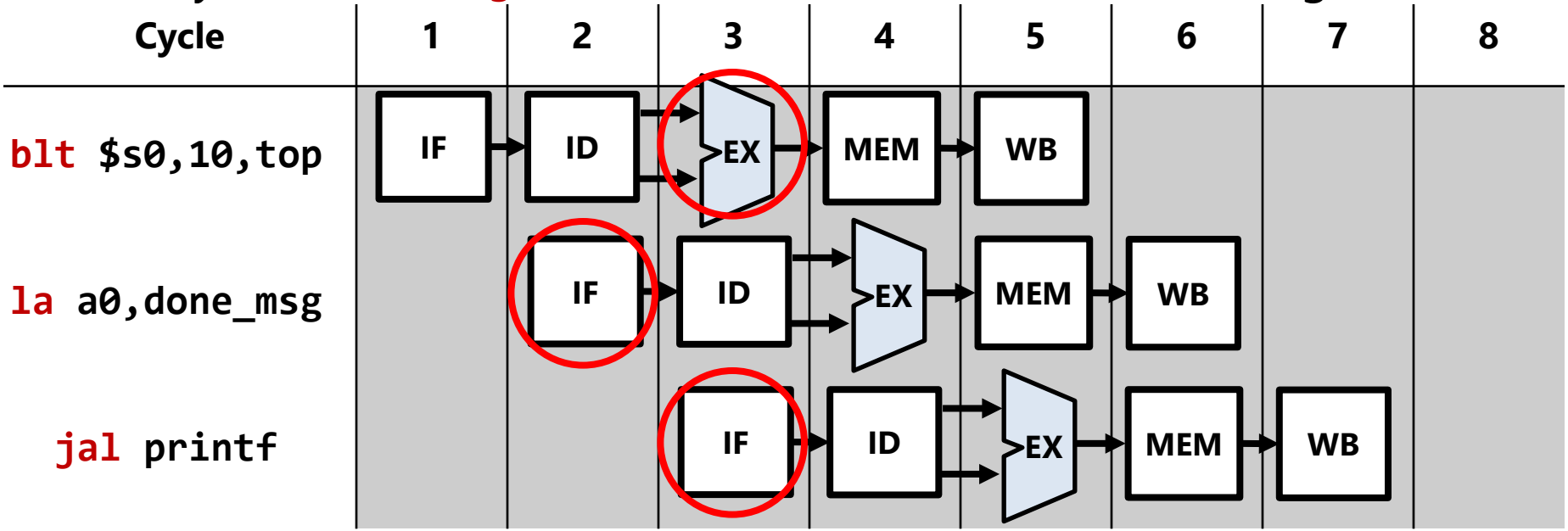
Hardware avoiding a control hazard

- Let's watch the previous example.



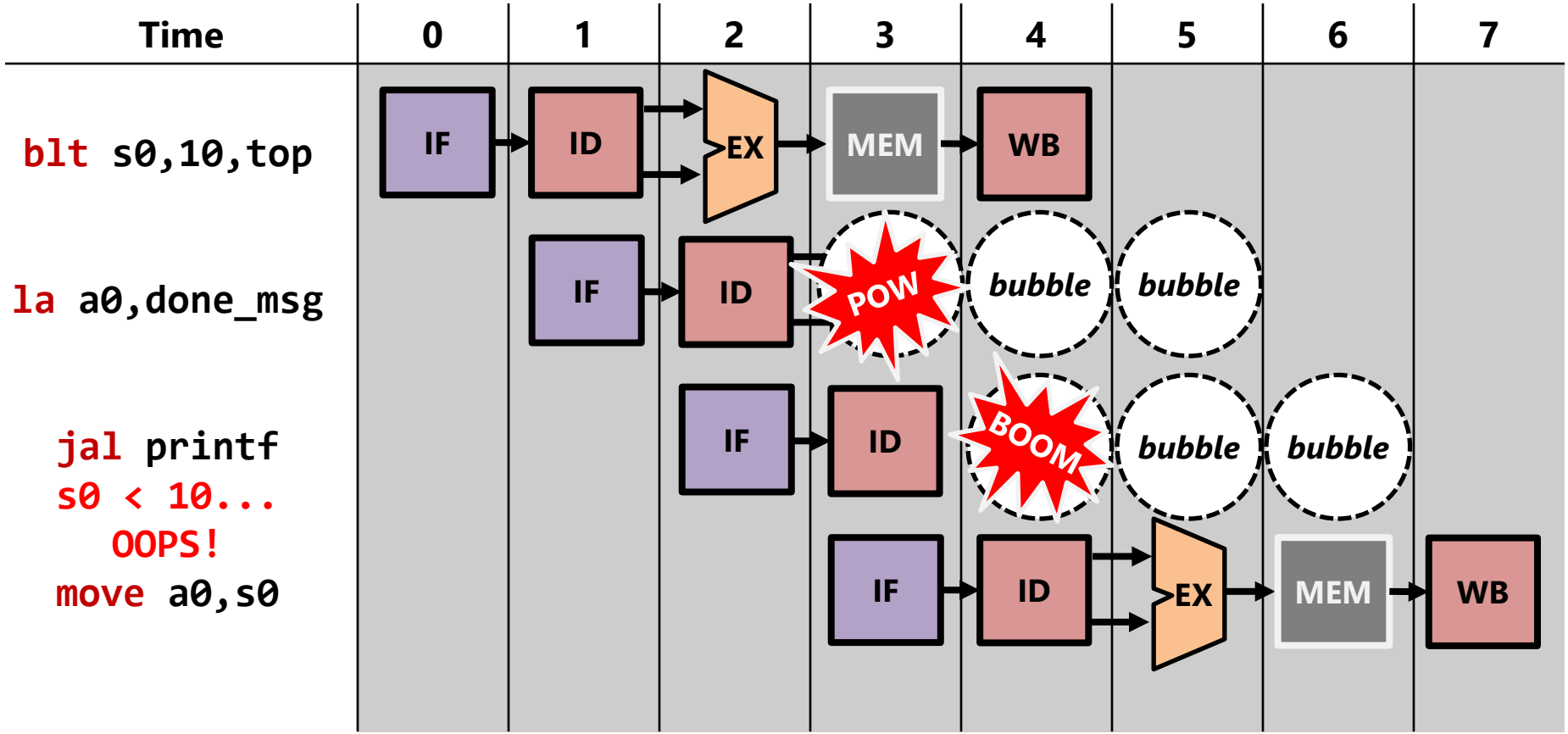
Hardware avoiding a control hazard

- Already fetched **la**, **jal** but later discovered it's the wrong branch.

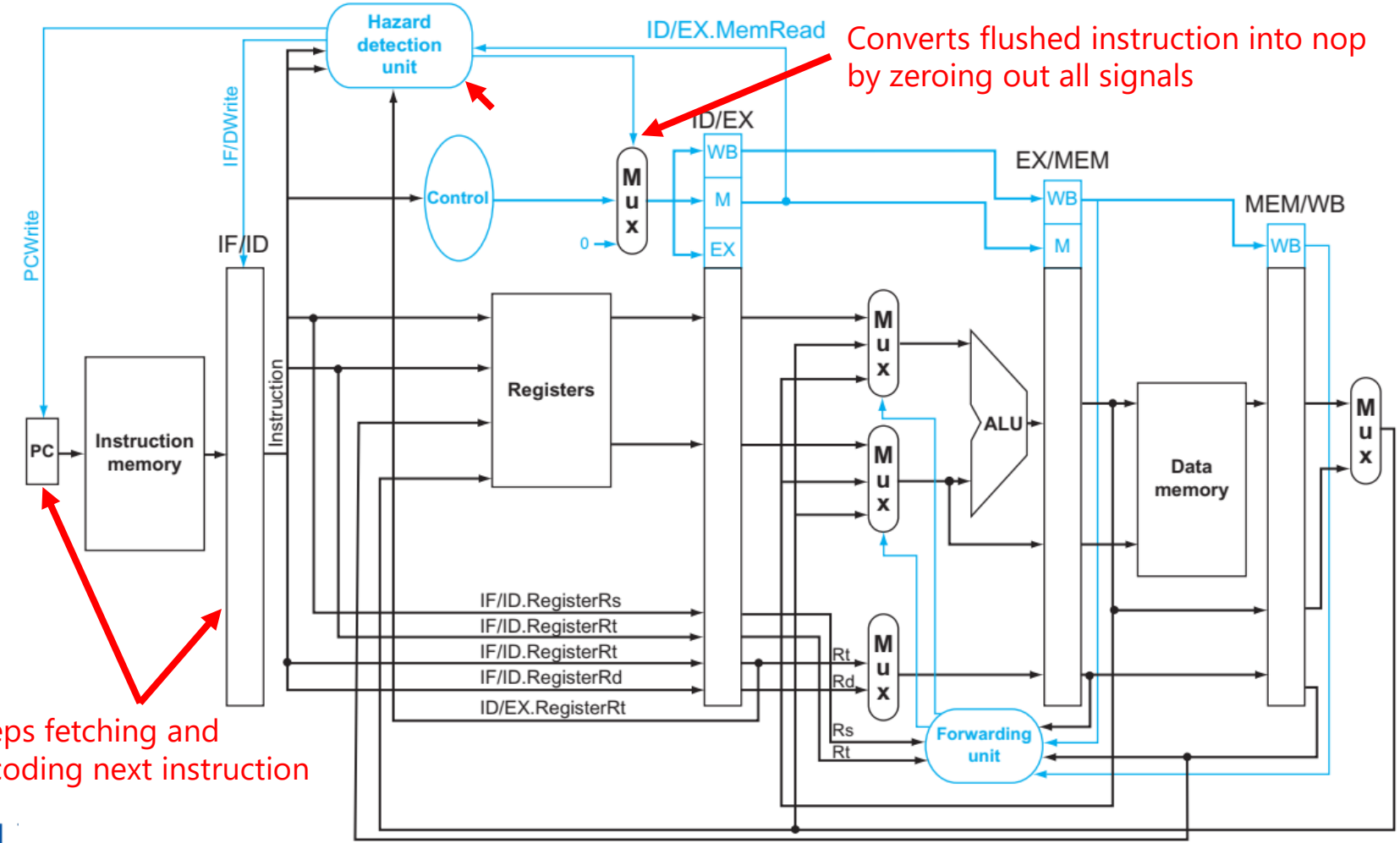


Hardware avoiding a control hazard

- A pipeline flush removes all wrong path instructions from pipeline



Hazard Detection Unit (HDU) on control hazard



Keeps fetching and decoding next instruction

Control Hazards cause flushes

- If a control hazard is detected in EX stage due to a branch:
  - Immediately starts fetching next instruction from branch target
  - Any "newer" instructions (those already in IF and ID stages) → transformed into **nops.**
  - Any "older" instructions (those in MEM and WB stages) → left to finish executing as normal.

Performance penalty of pipeline stalls

- Remember the three components of performance:

$$\frac{\text{instructions}}{\text{program}} \times \frac{\text{cycles}}{\text{instruction}} \times \frac{\text{seconds}}{\text{cycle}}$$

Architecture	Instructions	CPI	Cycle Time (1/F)
Single-cycle	Same	1	5 ns
Ideal 5-stage pipeline	Same	1	1 ns
Pipeline w/ stalls	Same	> 1	1 ns

- Pipelining increases ***clock frequency*** proportionate to depth
- Stalls and flushes increase ***CPI*** (cycles per instruction)

- These stalls and flushes can be sometimes avoided.
  - Wait for it… it is coming up in the next chapter.

Compiler scheduling vs. Hardware scheduling

Compiler scheduling vs. Hardware scheduling

	Compiler-scheduled	Hardware-scheduled
Energy Efficiency	✓ • Scheduling happens off-line.	✗ • Hazard detection by HDU and scheduling done at runtime.
Binary Portability	✗ • ISA must include assumptions about pipeline design • Must update ISA on pipeline re-design	✓ • ISA free of pipeline design assumptions • Can re-design pipeline maintaining the same ISA
Quality of Scheduling	△ • Pro: No limit to scheduling scope (full view of code) • Con: No runtime info. Hard to predict hazards. Hard to predict delay of memory instructions (due to caching).	△ • Pro: Can know exactly presence of hazards and instruction delays at runtime. • Con: Scheduling scope limited by size of hardware instruction scheduling queue